

UNIVERSIDAD REGIONAL AUTÓNOMA DE LOS ANDES (UNIANDES)

Departamento de Desarrollo de Software | Facultad de Sistemas

Programación de Backend y MCP en Python para IA Generativa

Estudiante: Juan Carlos Lozada | **Fecha:** 2 de septiembre de 2026 (Miércoles)

Informe de Entrega — Practica 3 / Sesion 4

Curso: Programacion de Backend y MCP en Python para IA Generativa

Tema: Spec Sencilla + Spec Kit

Estudiante: Juan Carlos Lozada

Institucion: Universidad Regional Autonoma de Los Andes (UNIANDES)

Departamento: Desarrollo de Software

Fecha de ejecucion: 13 de septiembre de 2026

1. Resumen de la Sesion

En esta practica se desarrollo un componente de backend para la conversion de unidades termicas (Celsius, Fahrenheit y Kelvin), comparando dos aproximaciones de ingenieria guiadas por especificaciones:

1. **Spec a Mano (`clase-sdd`)**: Redaccion manual estructurada con Objetivo, Criterios de Aceptacion cuantificables y Casos Borde.
2. **Spec Kit (`mi-proyecto-speckit`)**: Ciclo completo asistido mediante el framework de especificaciones (`/speckit-specify` -> `/speckit-plan` -> `/speckit-tasks` -> `/speckit-implement`).

El objetivo consistio en evaluar como la precision formal de una especificacion elimina la ambigüedad del *vibe coding* observado en la Sesion 3.

2. Bloque 3.A: Implementacion con Spec a Mano (`clase-sdd`)

2.1 Especificacion Manual (`spec_manual.md`)

Se redacto una especificacion delimitada que impone:

- * Conversion bidireccional exacta entre C, F y K.
- * Restriccion rigurosa del cero absoluto (rechazo de $T < 0\text{ K}$, $T < -273.15^{\circ}\text{C}$, $T < -459.67^{\circ}\text{F}$).
- * Redondeo obligatorio a dos decimales.
- * Manejo explicito de entradas no numericas y casos identidad.

2.2 Verificacion de Suite Automatizada

Se estructuro una suite de 7 pruebas unitarias con `pytest` :

```
tests/test_converter.py::test_conversion_celsius_a_fahrenheit PASSED [ 14%]
tests/test_converter.py::test_conversion_fahrenheit_a_celsius PASSED [ 28%]
tests/test_converter.py::test_conversion_celsius_a_kelvin PASSED [ 42%]
tests/test_converter.py::test_conversion_kelvin_a_celsius PASSED [ 57%]
tests/test_converter.py::test_misma_unidad PASSED [ 71%]
tests/test_converter.py::test_rechazo_debajo_cero_absoluto PASSED [ 85%]
tests/test_converter.py::test_tipo_no_numerico PASSED [100%]
===== 7 passed in 0.02s =====
```

3. Bloque 3.B: Implementacion con Spec Kit (`mi-proyecto-speckit`)

3.1 Flujo y Artefactos Producidos

A traves del flujo de Spec Kit se originaron los siguientes componentes:

- * `specs/temp-converter/spec.md` : Especificacion formal con Requerimientos Funcionales (FR-001 a FR-004), Escenarios de Aceptacion formateados como *Given / When / Then*, casos de borde y clarificaciones tecnicas.
- * `plan.md` : Diseno desacoplado en tres capas:
- * Motor de calculo matematico puro (`converter.py`).
- * Capa de servicio e integracion (`service.py`).
- * Interfaz de consola (`cli.py`).
- * `tasks.md` : Secuencia trazable de tareas (T001 - T008) desde el setup de dependencias hasta la ejecucion de pruebas.
- * **Suite de pruebas de aceptacion:** 9 pruebas unitarias verificando cada escenario Given/When/Then.

```
tests/test_converter.py::test_scenariio_1_celsius_to_fahrenheit PASSED [ 11%]
tests/test_converter.py::test_scenariio_2_fahrenheit_to_celsius PASSED [ 22%]
tests/test_converter.py::test_scenariio_3_celsius_to_kelvin PASSED [ 33%]
tests/test_converter.py::test_scenariio_4_identity_conversion PASSED [ 44%]
tests/test_converter.py::test_scenariio_5_absolute_zero_boundary PASSED [ 55%]
tests/test_converter.py::test_scenariio_6_rejection_below_absolute_zero PASSED [ 66%]
tests/test_converter.py::test_edge_cases_non_numeric PASSED [ 77%]
tests/test_converter.py::test_edge_cases_whitespace_and_invalid_unit PASSED [ 88%]
tests/test_converter.py::test_edge_cases_nan_and_inf PASSED [100%]
===== 9 passed in 0.02s =====
```

4. Evaluacion Comparativa

4.1 Casos de Prueba Lado a Lado

Tipo de Caso	Entrada	Resultado en Spec a Mano	Resultado en Spec Kit	Observacion Critica
Caso Normal	100 °C a F	212.0	212.0	Ambos cumplieron la regla de calculo sin inconvenientes.
Caso Borde	-273.15 °C a K	0.0	0.0	Ambos respetaron el limite exacto del cero absoluto.
Caso No Contemplado	<code>float('inf')</code>	Devuelve <code>inf</code>	Lanza <code>ValueError</code>	Spec Kit detecto la necesidad de blindar contra valores no finitos (<code>inf</code> , <code>nan</code>), mientras que la spec a mano paso por alto este detalle.

4.2 Matriz de Evaluacion

Criterio	Spec a Mano	Spec Kit
Cobertura de Casos Borde	Basica (limites fisicos y tipos)	Exhaustiva (limites fisicos, tipos, cadenas con espacios, valores no finitos)
Arquitectura Resultante	Archivo monolitico	Estructura modular desacoplada (<code>core</code> , <code>service</code> , <code>cli</code>)
Velocidad de Inicio	Inmediata (~5 minutos)	Requiere planificacion inicial (~15 minutos)
Confiabilidad para Produccion	Media	Muy Alta

5. Conclusiones y Frase de Cierre

- Frase de Sintesis:**

"La proxima vez que tenga un proyecto de tamano mediano o grande, elegiria Spec Kit porque establece una arquitectura modular con trazabilidad completa desde los requerimientos funcionales (FR) hasta las pruebas automatizadas, evitando que se omitan casos borde criticos de seguridad y consistencia en el backend."

- La metodologia *Spec-Driven Development* traslada el esfuerzo del programador hacia la definicion rigurosa de contratos y pruebas, garantizando que el asistente de IA construya software deterministicamente verificable.